Experiment No-27 : Best First Search Algorithm

Aim:

To write a Prolog program to implement the Best First Search algorithm.

Programming Code:

```
% h(Node, Value): heuristic (estimated cost from the node to the goal g)
h(s, 10).
h(a, 7).
h(b, 5).
h(c, 8).
h(d, 6).
h(e, 3).
h(f, 4).
h(g, 0).

% Edges of the graph
edge(s, a).
edge(s, b).
edge(a, c).
edge(a, d).
edge(b, e).
edge(b, f).
edge(e, g).

connected(X, Y) :- edge(X, Y).
connected(X, Y) :- edge(Y, X).

% best_first(Start, Goal, Path)
best_first(Start, Goal, Path) :-
    h(Start, H),
    bfs([H-[Start]], Goal, RevPath),
    reverse(RevPath, Path).

% The open list holds H-Path pairs, sorted so that the best (lowest H) is first
bfs([_-[Goal|Rest]|_], Goal, [Goal|Rest]) :-
    format("Reached goal node ~w~n", [Goal]), !.
bfs([_-[Node|Rest]|Open], Goal, Path) :-
    h(Node, HN),
    format("Expanding node ~w (h = ~w)~n", [Node, HN]),
    findall(H-[Next, Node|Rest],
            ( connected(Node, Next),
              \+ memberchk(Next, [Node|Rest]),
              h(Next, H) ),
            Children),
    append(Open, Children, NewOpen),
    keysort(NewOpen, Sorted),
    bfs(Sorted, Goal, Path).
```

OUTPUT:

```
$ swipl -q exp27.pl

?- best_first(s, g, Path).
Expanding node s (h = 10)
Expanding node b (h = 5)
Expanding node e (h = 3)
Reached goal node g
Path = [s,b,e,g].

?- best_first(s, d, Path).
Expanding node s (h = 10)
Expanding node b (h = 5)
Expanding node e (h = 3)
Expanding node g (h = 0)
Expanding node f (h = 4)
Expanding node a (h = 7)
Reached goal node d
Path = [s,a,d].

?- best_first(s, c, Path).
Expanding node s (h = 10)
Expanding node b (h = 5)
Expanding node e (h = 3)
Expanding node g (h = 0)
Expanding node f (h = 4)
Expanding node a (h = 7)
Expanding node d (h = 6)
Reached goal node c
Path = [s,a,c].

?- best_first(a, g, Path).
Expanding node a (h = 7)
Expanding node d (h = 6)
Expanding node c (h = 8)
Expanding node s (h = 10)
Expanding node b (h = 5)
Expanding node e (h = 3)
Reached goal node g
Path = [a,s,b,e,g].

$
```

Result

The Best First Search algorithm was implemented successfully using Prolog. At every step the program expands the node with the lowest heuristic value from the open list until the goal node is reached, and returns the path found.